

PMLZA 5. Bayesian Neural Nets

UoP

David Rios Insua

August 2026

ICMAT-CSIC

Objectives

Introduce key concepts about neural networks, from shallow to deep.

Mention some currently important architectures: convolutional, (recurrent), transformer, autoencoder, GANS.

Introduce computational strategies

Sketch some of their major application domains. Introduce stochastic gradient descent and variants.

Contents

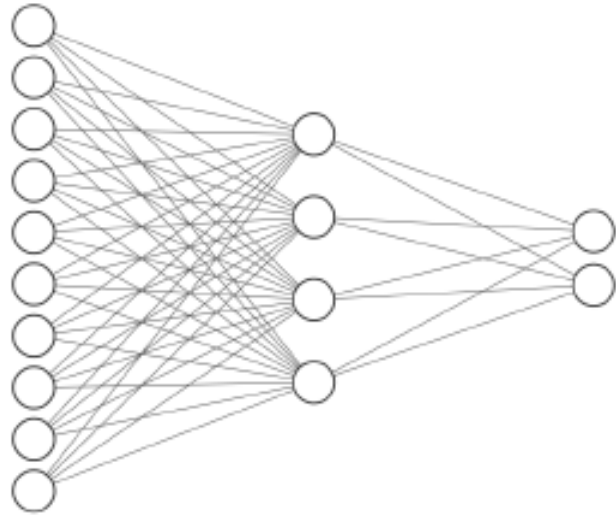
- Introduction
- (Shallow) neural networks
- Deep neural networks
- Specific architectures
- Computational strategies

NNs. Motivation

Motivation

- AI is ultra cool because of deep learning
- ML is very cool because of deep learning
- Stats is pretty cool because of deep learning
- Lots of jobs around.... but everyone claims to know about DL...
- Many exciting research questions
- Many exciting computational problems

Concept



Input Layer $\in \mathbb{R}^8$

Hidden Layer $\in \mathbb{R}^4$

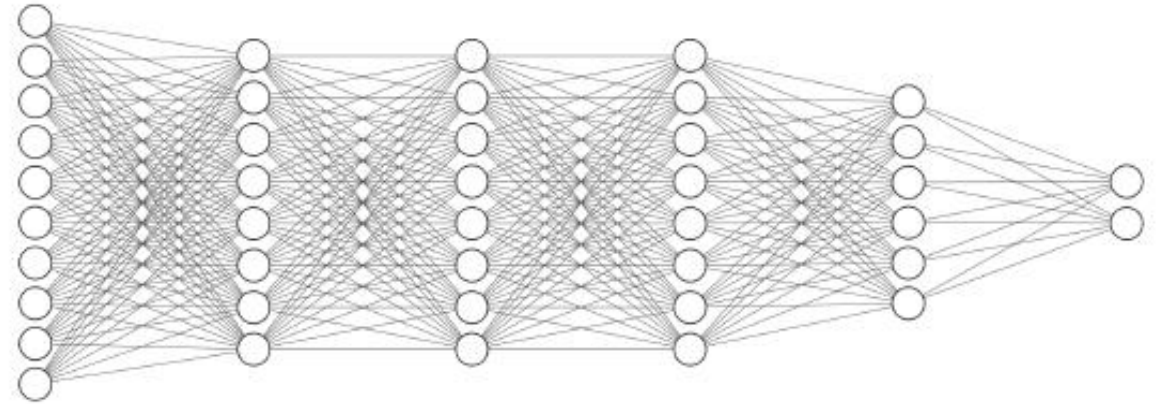
Output Layer $\in \mathbb{R}^2$

$$y = \sum_{j=1}^m \beta_j \psi(x' \gamma_j) + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

$$\psi(\eta) = \exp(\eta) / (1 + \exp(\eta))$$

(Shallow) Neural nets



$$\{f_0, f_1, \dots, f_{L-1}\}$$

$$z_{l+1} = f_l(z_l, \gamma_l).$$

$$y = \sum_{j=1}^{m_L} \beta_j z_{L,j} + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

Deep neural nets

Some nice apps for better understanding

<https://playground.tensorflow.org/>

<https://www.i-am.ai/neural-numbers.html>

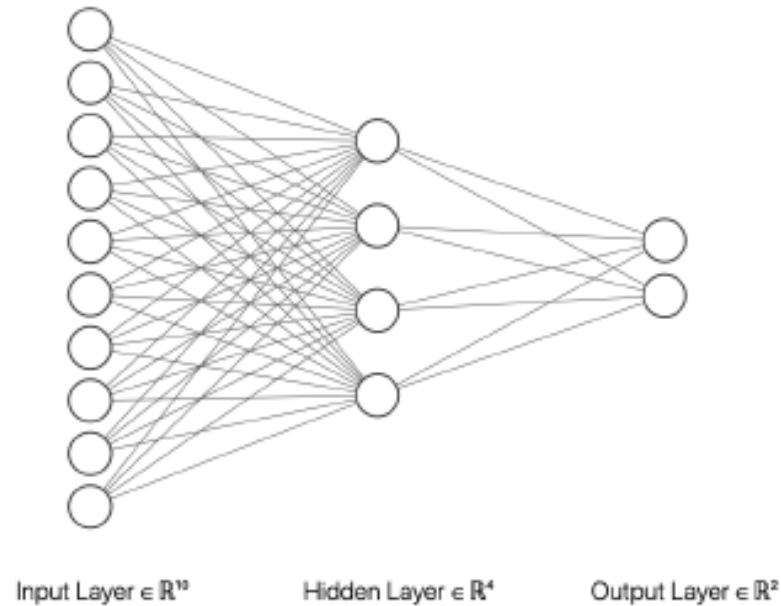
<https://www.i-am.ai/piano-genie.html>

<http://places2.csail.mit.edu/>

<https://modeldepot.github.io/tfjs-yolo-tiny-demo/>

Shallow neural nets

Formulation



$$y = \sum_{j=1}^m \beta_j \psi(x' \gamma_j) + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

$$\psi(\eta) = \exp(\eta) / (1 + \exp(\eta))$$

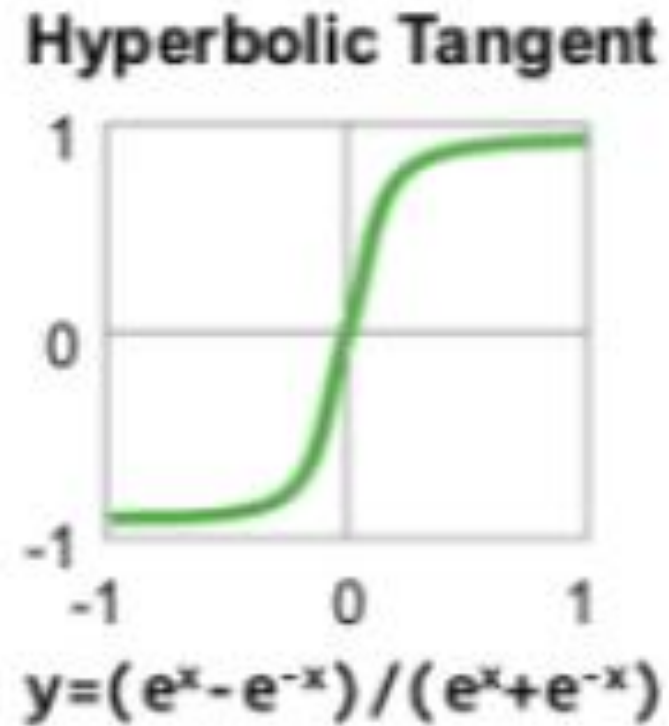
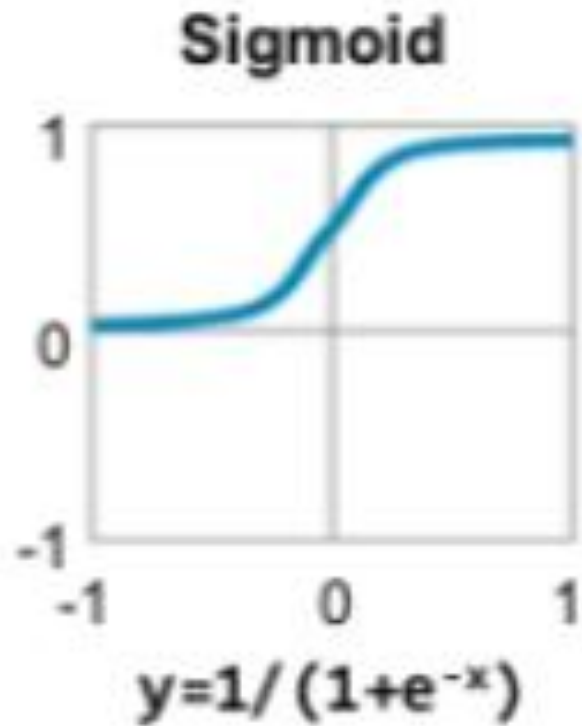
Linear in β 's, nonlinear in γ 's

Motivation. Cybenko's theorem

Any continuous function in the r -dimensional cube may be approximated by models of type $\sum_{j=1}^m \beta_j \psi(x' \gamma_j)$ when the activation function is sigmoidal

(as m goes to infinity)

Early activation functions (logistic and tanh)



Training

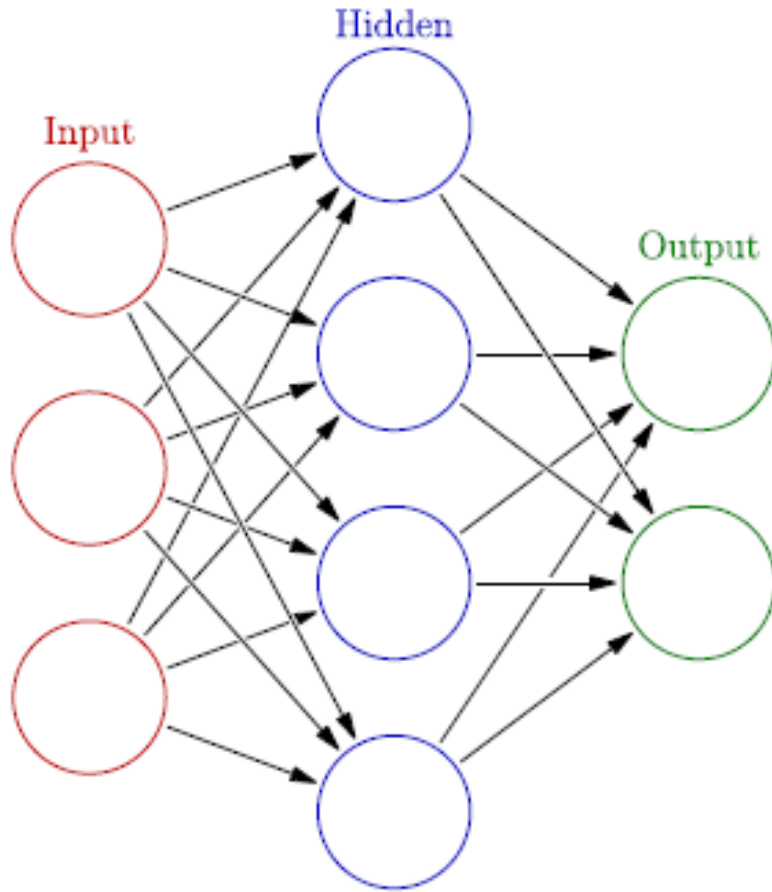
Given training data, maximise log-likelihood

$$\min_{\beta, \gamma} f(\beta, \gamma) = \sum_{i=1}^n f_i(\beta, \gamma) = \sum_{i=1}^n \left(y_i - \sum_{j=1}^m \beta_j \psi(x_i' \gamma_j) \right)^2$$

Gradient descent

Backpropagation to estimate gradient

Example with Keras



```
library(keras)

model <- keras_model_sequential()

# arquitectura
model %>%
  layer_dense(units = 4,
              activation = 'sigmoid',
              input_shape = c(3)) %>%
  layer_dense(units = 2,
              activation = 'linear')

# definir entrenamiento
model %>% compile(loss = "mse",
                 optimizer = optimizer_sgd())

# entrenamiento
model %>% fit(X_train, y_train,
             epochs = 10, batch_size = 128,
             validation_size = 0.2)

# error de test
model %>% evaluate(X_test)
```

Training with regularisation

$$\min_{\beta, \gamma} f(\beta, \gamma) = \sum_{i=1}^n f_i(\beta, \gamma) = \sum_{i=1}^n \left(y_i - \sum_{j=1}^m \beta_j \psi(x'_i \gamma_j) \right)^2$$

$$\min g(\beta, \gamma) = f(\beta, \gamma) + h(\beta, \gamma),$$

Weight decay

$$h(\beta, \gamma) = \lambda_1 \sum \beta_i^2 + \lambda_2 \sum \sum \gamma_{ji}^2$$

Ridge

Bayesian analysis of shallow neural nets

Recall MLE+regularisation $\rightarrow$ MAP !!!!

Bayesian analysis of shallow neural nets (fixed arch)

$$y = \sum_{j=1}^m \beta_j \psi(\mathbf{x}' \gamma_j) + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

$$\psi(\eta) = \exp(\eta) / (1 + \exp(\eta))$$

$$\beta_i \sim N(\mu_\beta, \sigma_\beta^2) \text{ and } \gamma_i \sim N(\mu_\gamma, S_\gamma^2)$$

$$\mu_\beta \sim N(a_\beta, A_\beta), \mu_\gamma \sim N(a_\gamma, A_\gamma), \sigma_\beta^{-2} \sim \text{Gamma}(c_b/2, c_b C_b/2)$$

$$S_\gamma^{-1} \sim \text{Wish}(c_\gamma, (c_\gamma C_\gamma)^{-1}) \text{ and } \sigma^{-2} \sim \text{Gamma}(s/2, sS/2)$$

Bayesian analysis of shallow neural nets (fixed arch)

```
1 Start with arbitrary  $(\beta, \gamma, \nu)$ .
2 while not convergence do
3   Given current  $(\gamma, \nu)$ , draw  $\beta$  from  $p(\beta|\gamma, \nu, y)$  (a multivariate normal).
4   for  $j = 1, \dots, m$ , marginalizing in  $\beta$  and given  $\nu$  do
5     Generate a candidate  $\tilde{\gamma}_j \sim g_j(\gamma_j)$ .
6     Compute  $a(\gamma_j, \tilde{\gamma}_j) = \min\left(1, \frac{p(D|\tilde{\gamma}, \nu)}{p(D|\gamma, \nu)}\right)$  with  $\tilde{\gamma} = (\gamma_1, \gamma_2, \dots, \tilde{\gamma}_i, \dots, \gamma_m)$ .
7     With probability  $a(\gamma_j, \tilde{\gamma}_j)$  replace  $\gamma_j$  by  $\tilde{\gamma}_j$ . If not, preserve  $\gamma_j$ .
8   end
9   Given  $\beta$  and  $\gamma$ , replace  $\nu$  based on their posterior conditionals:
10   $p(\mu_\beta|\beta, \sigma_\beta)$  is normal;  $p(\mu_\gamma|\gamma, S_\gamma)$ , multivariate normal;  $p(\sigma_\beta^{-2}|\beta, \mu_\beta)$ ,
    Gamma;  $p(S_\gamma^{-1}|\gamma, \mu_\gamma)$ , Wishart;  $p(\sigma^{-2}|\beta, \gamma, y)$ , Gamma.
11 end
```


Bayesian analysis of shallow neural nets (var arch)

$$\begin{aligned}y &= x_i' a + \sum_{j=1}^{m^*} d_j \beta_j \psi(x' \gamma_j) + \epsilon \\ \epsilon &\sim N(0, \sigma^2), \\ \psi(\eta) &= \exp(\eta) / (1 + \exp(\eta)), \\ \Pr(d_j = k | d_{j-1} = 1) &= (1 - \alpha)^{1-k} \times \alpha^k, k \in \{0, 1\} \\ \beta_i &\sim N(\mu_b, \sigma_\beta^2), a \sim N(\mu_a, \sigma_a^2), \gamma_i \sim N(\mu_\gamma, \Sigma_\gamma).\end{aligned}$$

Reversible jump algo

Classification

Use a multinomial likelihood

$$p(y|x, \beta, \gamma) = \text{Mult}(n = 1, p_1(x, \beta, \gamma), \dots, p_K(x, \beta, \gamma)),$$

Use softmax to compute class probabilities

$$p_k = \frac{\exp\{\beta_k \psi(x' \gamma_k)\}}{\exp\left\{\sum_{k=1}^K \beta_k \psi(x' \gamma_k)\right\}}$$

Other

Non-linear autoregression

Semi-parametric regression

(Gaussian process)

$$y = \sum_{j=1}^m \beta_j \psi(x' \gamma_j) + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

$$\psi(\eta) = \exp(\eta) / (1 + \exp(\eta))$$

Deep neural nets

Hardware

Recall training NNs entails many, many, many matrix (tensor) multiplications

- From 1990 until 2010 CPUs speed increases by factor 5000
- NNs trainable on standard laptops (still insufficient for CNNs, RNNs,...)
- GPUs https://en.wikipedia.org/wiki/Graphics_processing_unit
 - Simpler processors
 - Useful for processing large data blocks in parallel. Parallelisation
- Nvidia Titan X (app 1000€) 350 times more powerful than modern laptop
6,6 trillion operations in floating comma per second
- TPUs... https://en.wikipedia.org/wiki/Tensor_Processing_Unit

Data

- Exponential development in storage technology
- Internet explodes. New data generated, collected and distributed
 - Wikipedia (text)
 - YouTube (video)
 - Flickr (images)
 - Twitter (opinions)
 - Smartphone (geo)
 - ...
- Benchmark competitions: Imagenet, Kaggle,...

Algos

- Until 2010 no reliable training algos with NNs (beyond 1 or 2 hidden layers)
 - Vanishing gradient
- Advances
 - New activation functions beyond sigmoidals
 - SGD recalled and important variants: ADAM et al
- ‘Easy’ to train networks of 10 or more layers
- Feasible now even in the 100’s layers,....

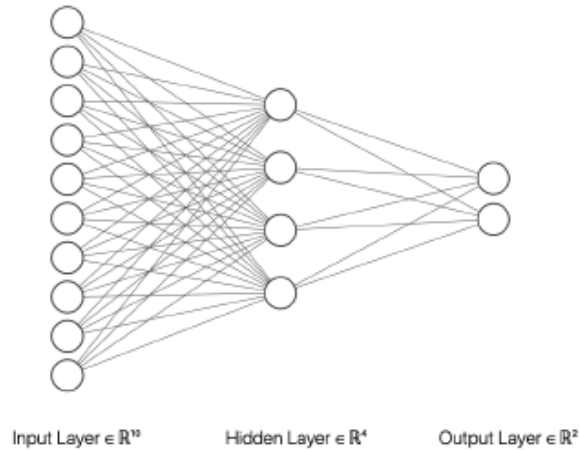
Software democratisation

- Before: programming GPUs was sophisticated (CUDA, C++)
<https://en.wikipedia.org/wiki/CUDA>
- From 2010 libraries like
 - Torch [https://en.wikipedia.org/wiki/Torch_\(machine_learning\)](https://en.wikipedia.org/wiki/Torch_(machine_learning)) replaced by PyTorch
 - Theano [https://en.wikipedia.org/wiki/Theano_\(software\)](https://en.wikipedia.org/wiki/Theano_(software)) now forked as Aesara
 - Caffe [https://en.wikipedia.org/wiki/Caffe_\(software\)](https://en.wikipedia.org/wiki/Caffe_(software))
 - TensorFlow <https://en.wikipedia.org/wiki/TensorFlow>

Facilitate automatic differentiation, matrix and tensor operations, take advantage of GPUs (and TPUs)

- On top of them Keras (interface to TensorFlow), PyTorch (Torch) facilitating their use

Concept

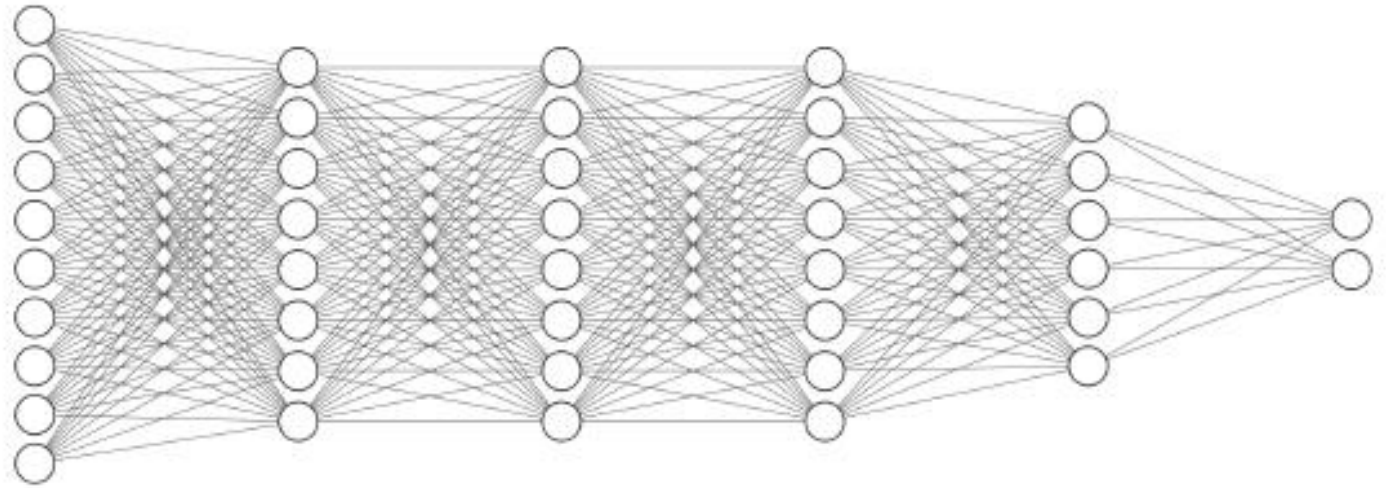


$$y = \sum_{j=1}^m \beta_j \psi(x' \gamma_j) + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

$$\psi(\eta) = \exp(\eta) / (1 + \exp(\eta))$$

(Shallow) Neural nets



$$\{f_0, f_1, \dots, f_{L-1}\}$$

$$z_{l+1} = f_l(z_l, \gamma_l).$$

$$y = \sum_{j=1}^{m_L} \beta_j z_{L,j} + \epsilon$$

$$\epsilon \sim N(0, \sigma^2),$$

Deep neural nets

Motivation. New universal approximation theorems

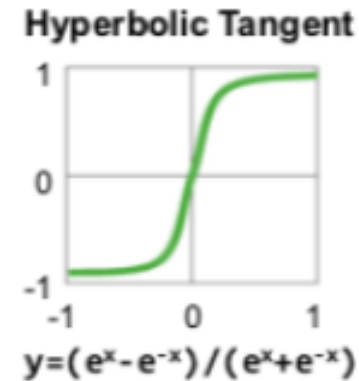
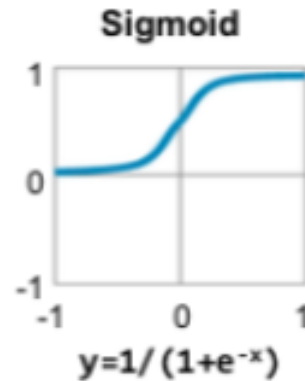
Arbitrary-width. Remember Cybenko's (and others)

Arbitrary-depth.

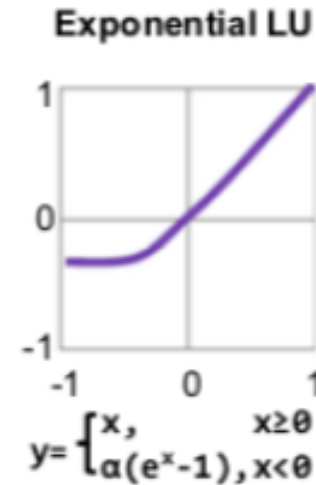
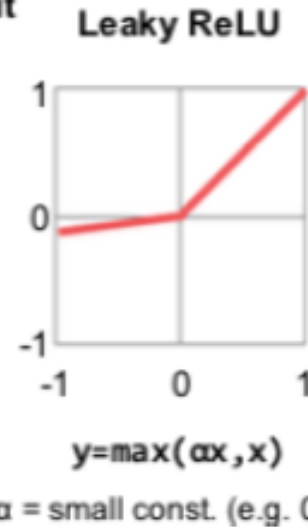
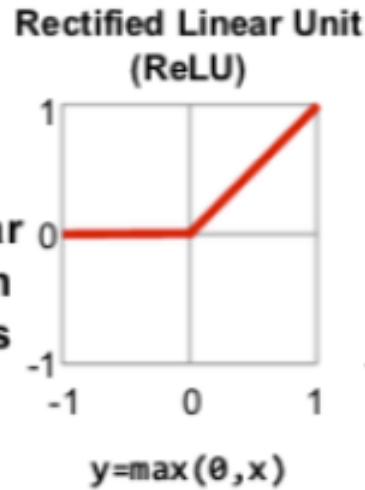
Any (Lebesgue) integrable function can be arbitrarily approximated by **RELU** fully-connected neural network by as its depth goes to infinity (and its width is bounded by $\max(n+1, m)$ n dimension of origin, m dimension of image)

The evolution in activation functions

Traditional
Non-Linear
Activation
Functions



Modern
Non-Linear
Activation
Functions



From gradient descent...

Training goes through minimising

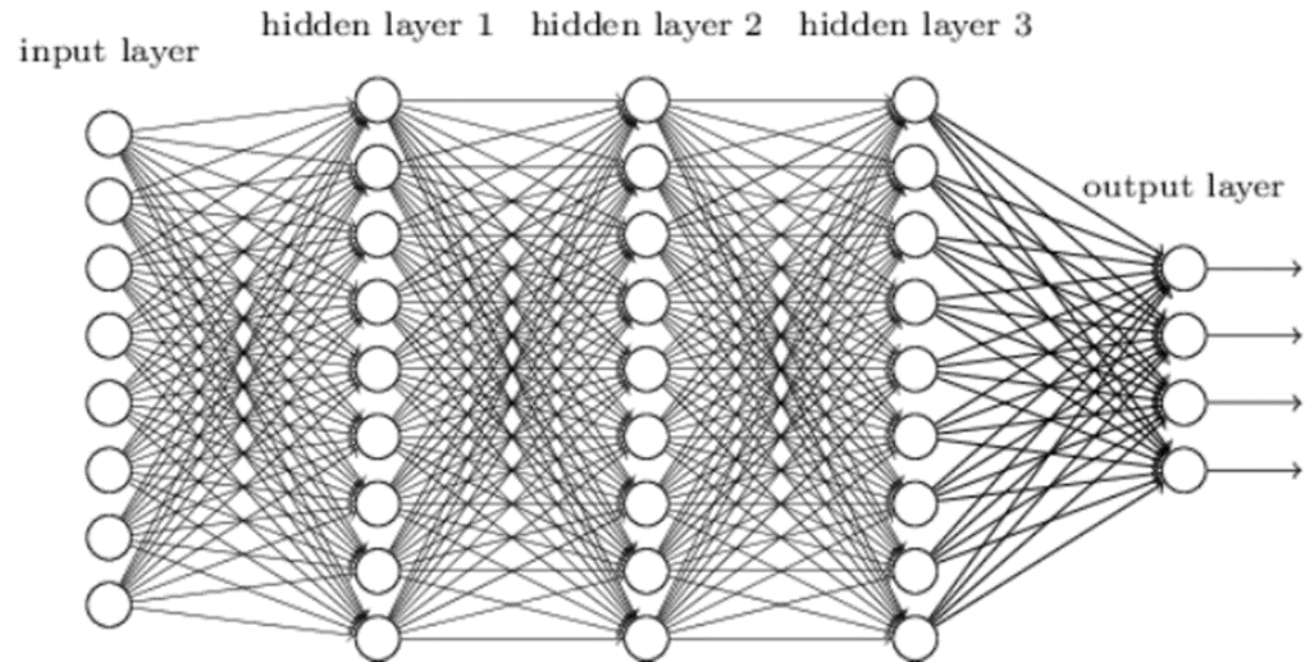
$$J(\theta) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \theta) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

$$L(\mathbf{x}, y, \theta) = -\log p(y \mid \mathbf{x}; \theta)$$

Requires gradient

$$\nabla_{\theta} J(\theta) = \frac{1}{m} \sum_{i=1}^m \nabla_{\theta} L(\mathbf{x}^{(i)}, y^{(i)}, \theta)$$

Might not even fit in memory, very slow anyway



... to stochastic gradient descent

(Randomly) sample a minibatch of size m' .

Approximate gradient

$$\mathbf{g} = \frac{1}{m'} \nabla_{\boldsymbol{\theta}} \sum_{i=1}^{m'} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta})$$

Update via gradient descent

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \epsilon \mathbf{g}$$

Stochastic gradient descent

Require: Learning rate ϵ_k .

Require: Initial parameter θ

while stopping criterion not met **do**

Sample a minibatch of m examples from the training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ with corresponding targets $\mathbf{y}^{(i)}$.

Compute gradient estimate: $\hat{\mathbf{g}} \leftarrow +\frac{1}{m} \nabla_{\theta} \sum_i L(f(\mathbf{x}^{(i)}; \theta), \mathbf{y}^{(i)})$

Apply update: $\theta \leftarrow \theta - \epsilon \hat{\mathbf{g}}$

end while

Use backprop
at this stage

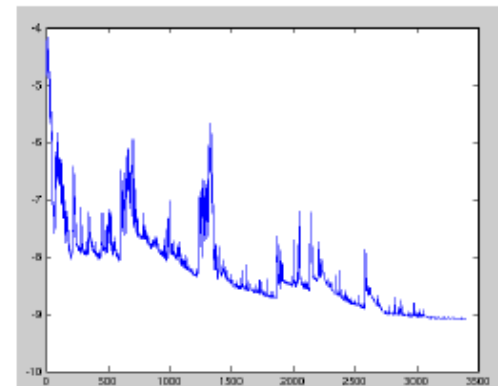
SGD. Robbins Monro conditions (1954!!!!)

If the learning parameters are chosen so that $\sum_{k=1}^{\infty} \epsilon_k = \infty$ and the gradient estimator is unbiased $\sum_{k=1}^{\infty} \epsilon_k^2 < \infty$ then SDG converges a.s. (to a local optimum)

NB: The minibatch size can even be 1 (some authors reserve SDG name to size 1 batches) , ie randomly sample just one instance and proceed

NB2: The batch now fits in memory!!!

NB3: May aid in escaping local minima!!!



Including regularizers

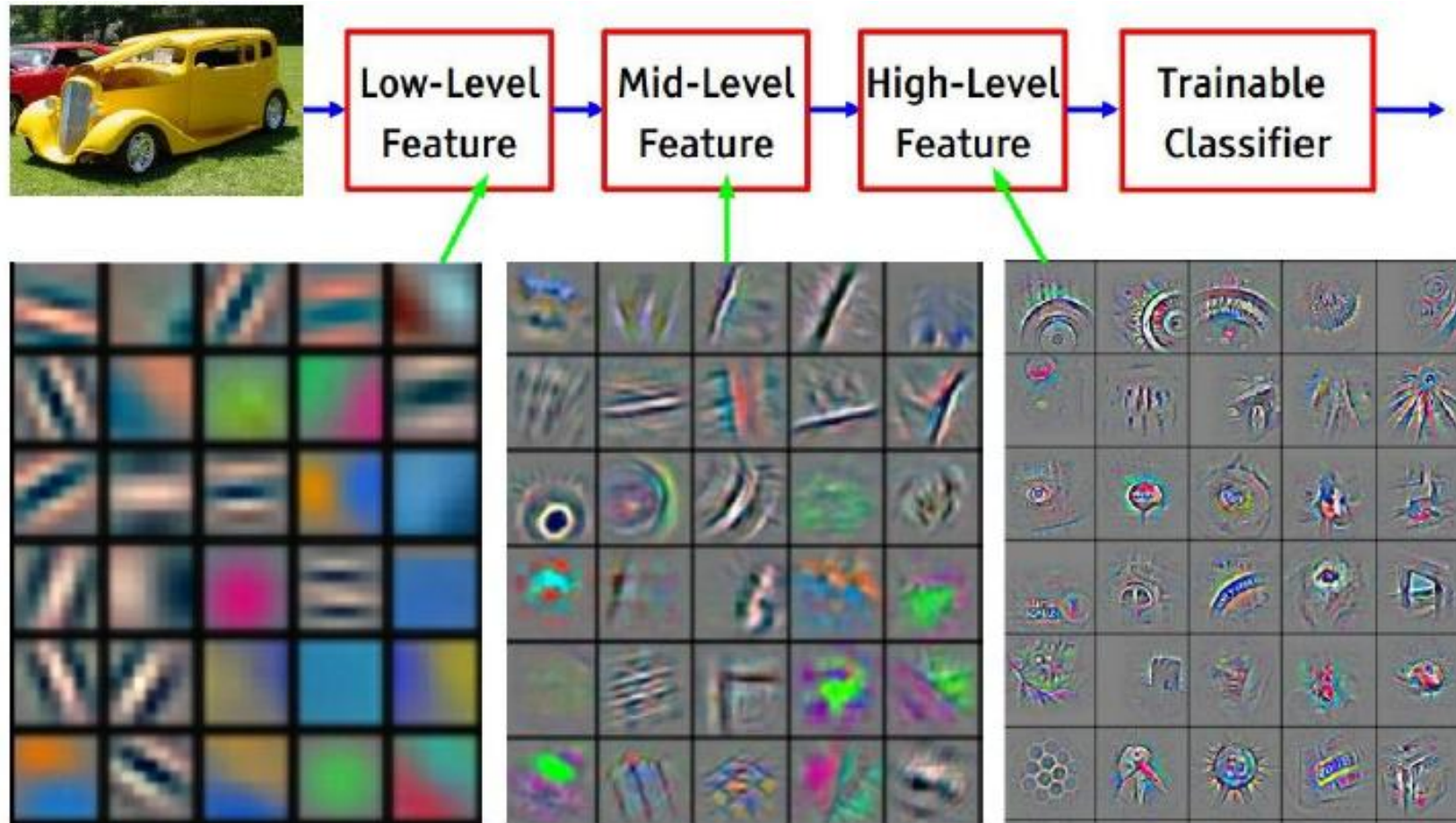
$$J(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}, y \sim \hat{p}_{\text{data}}} L(\mathbf{x}, y, \boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}) + h(\boldsymbol{\theta})$$

$$L(\mathbf{x}, y, \boldsymbol{\theta}) = -\log p(y \mid \mathbf{x}; \boldsymbol{\theta})$$

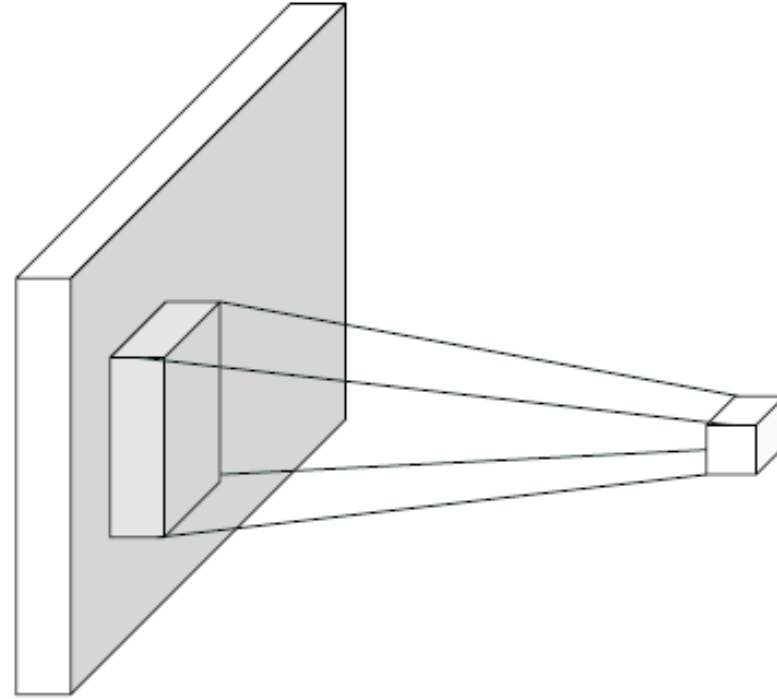
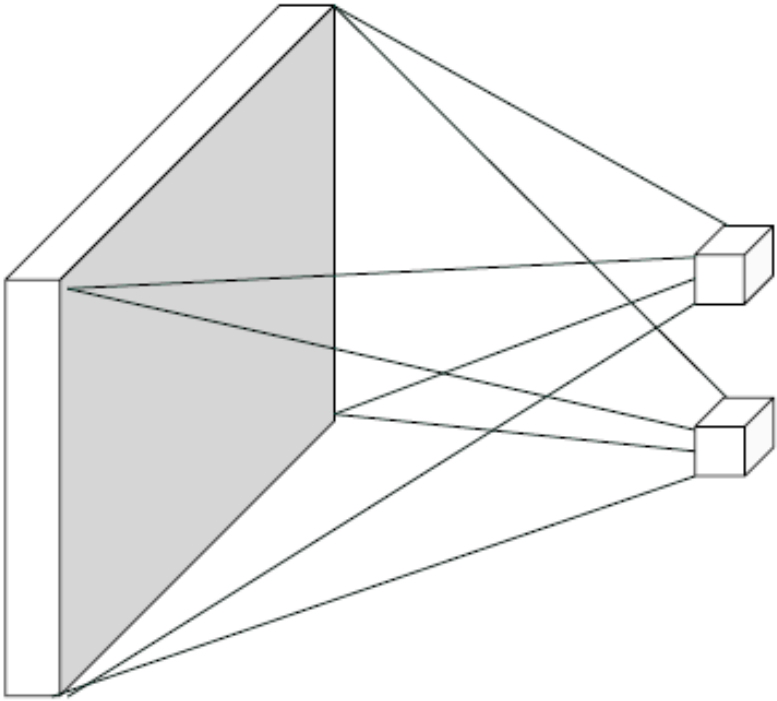
$$\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta}) = \frac{1}{m} \sum_{i=1}^m \nabla_{\boldsymbol{\theta}} L(\mathbf{x}^{(i)}, y^{(i)}, \boldsymbol{\theta}) + \nabla_{\boldsymbol{\theta}} h(\boldsymbol{\theta})$$

So same strategy works

CNNs. Typical structure

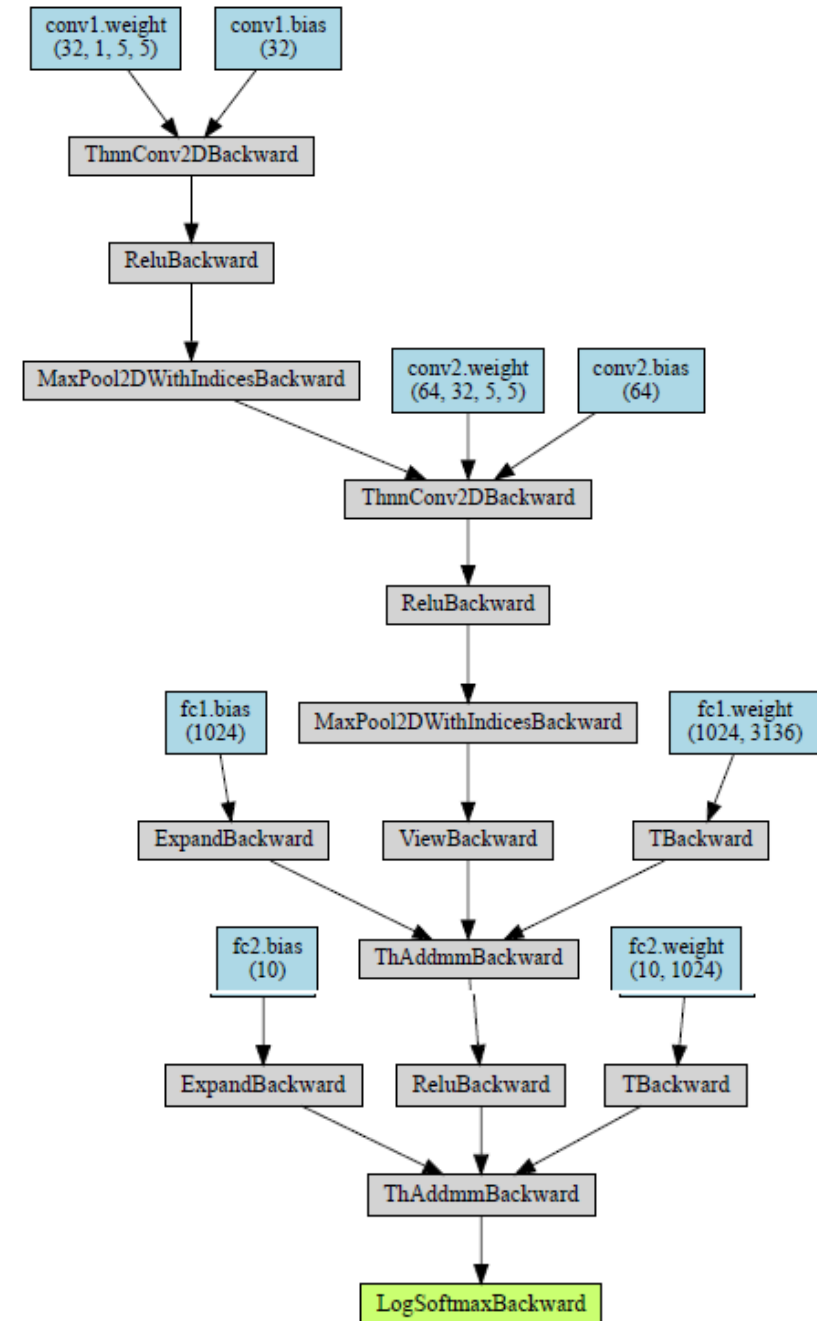


From fully connected DNs to convolutional (deep) nets

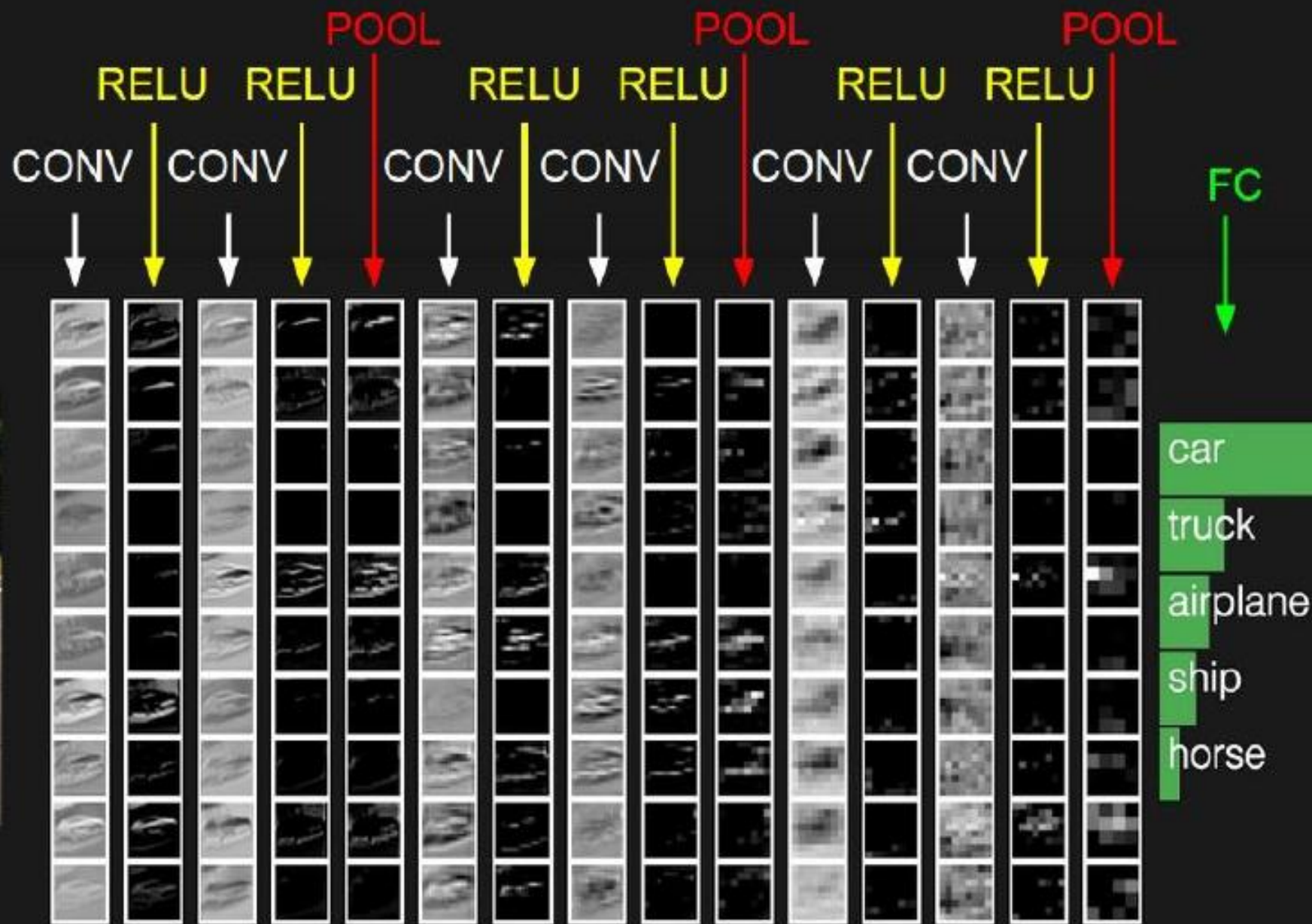


Towards convolutional networks

- Three concepts
 - Convolution
 - RELU
 - MaxPool
- Applications
 - Image recognition (e.g., security)
 - Video analysis (e.g., ADS)
 - Sound analysis
 - Pharma discovery
 - And others



CNNs. Typical structure



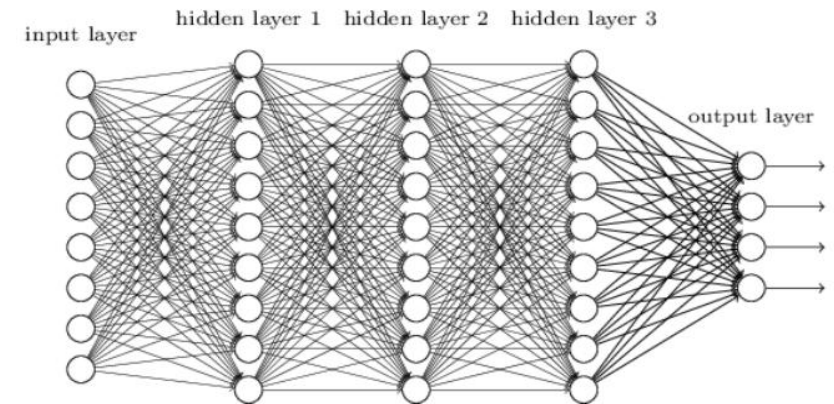
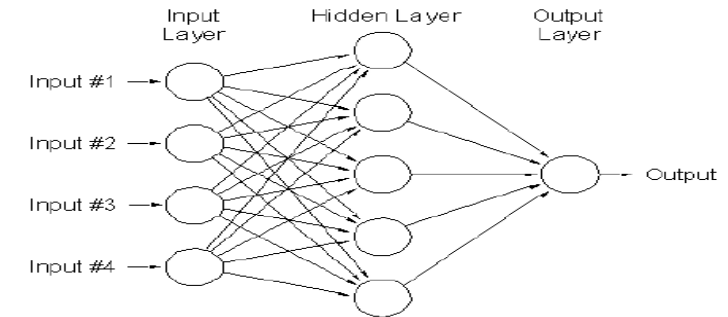
NNs for time series. Nonlinear autoregressions...

Shallow NN

$$y_j = \sum_{i=1}^m \beta_i \psi(x_k \omega_i) + \varepsilon_j$$

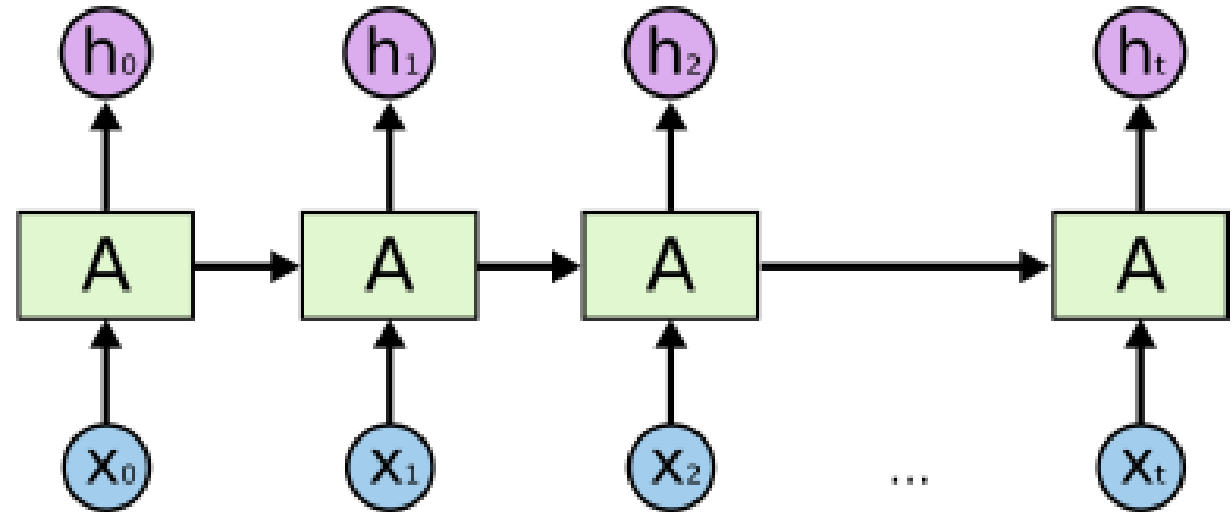
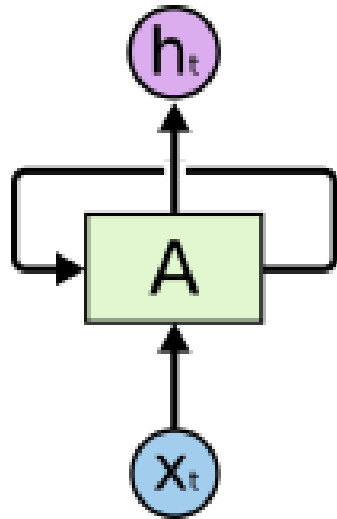
$$\min_{\beta, w} \sum_{k=1}^n \left(y_k - \sum_{i=1}^m \beta_i \psi(x_k \omega_i) \right)^2$$

- Input. Some entries, prior time series observations
- Output, value of time series to be forecast (one step ahead, two steps ahead,....)

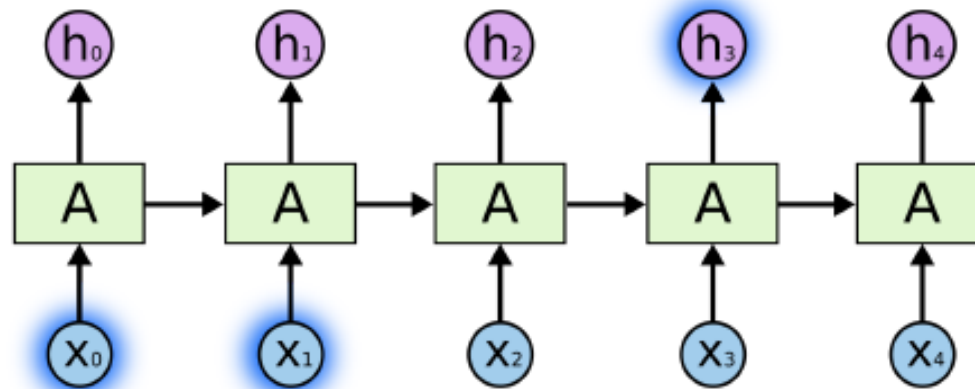


Fully connected... theoretically OK, but lot to be gained from special structure

RNNs

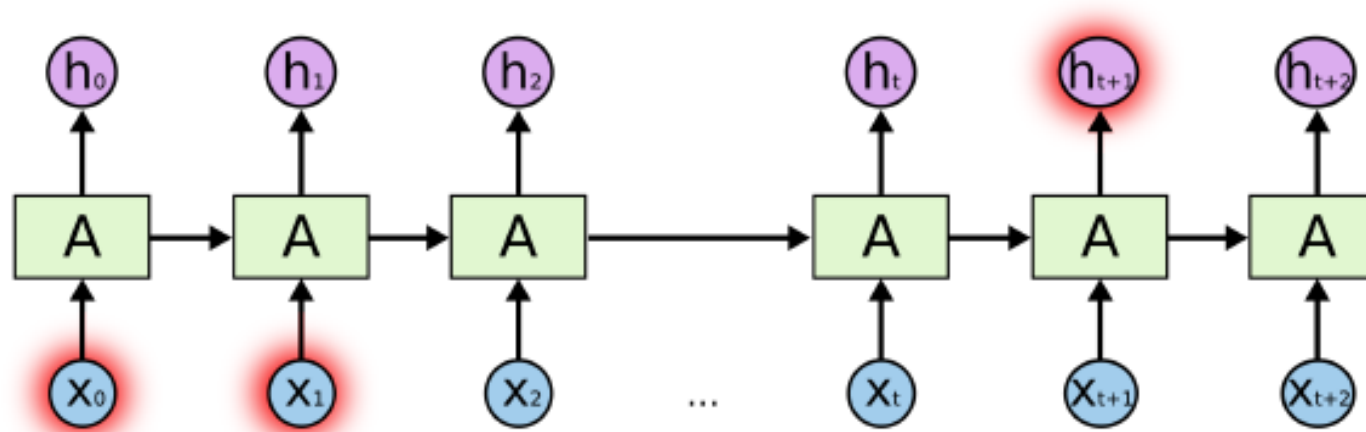


RNNs: Short-term vd Long-term dependencies



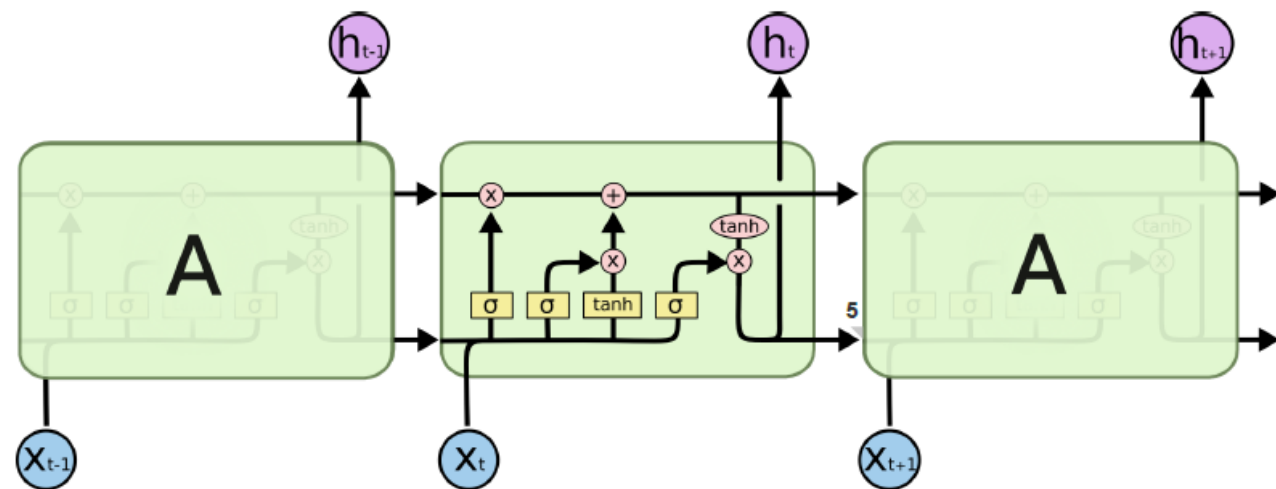
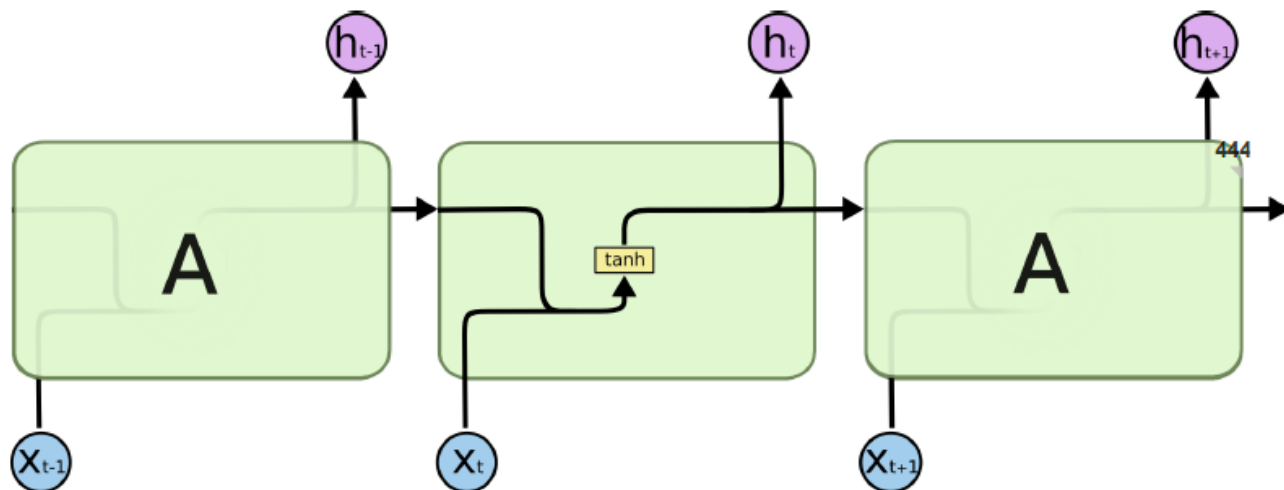
The clouds are in the

RNNs: Short-term vs Long-term dependencies

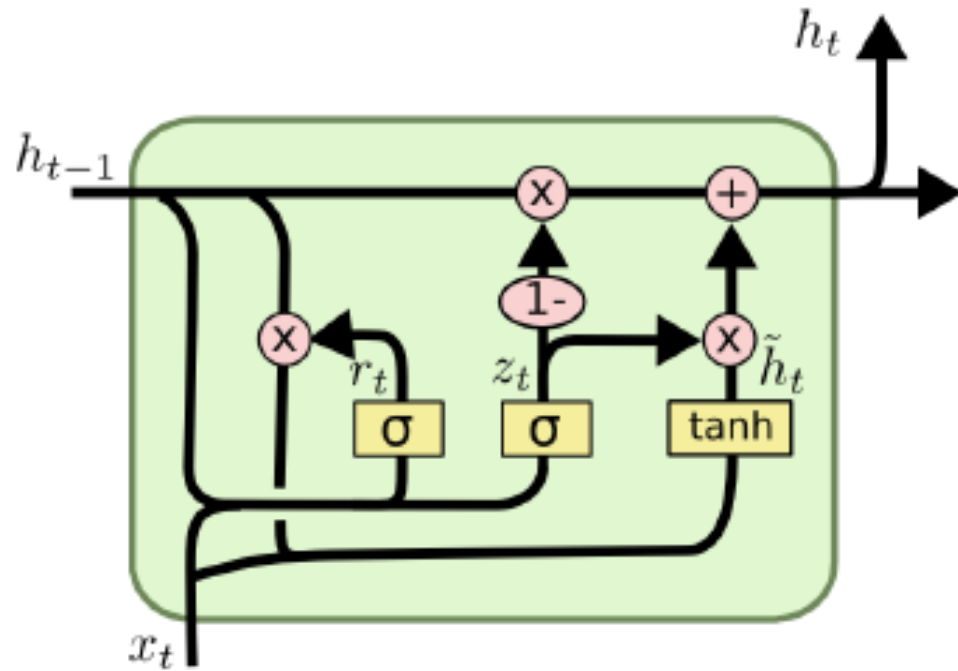


I grew up in France.... I speak fluent

From RNNs to LSTMs



Gate recurrent unit



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Word embeddings

Dense word vectors

Low-dimensional, floating point vectors

Learned from data

- Jointly with the main task. Embedding layer

- Precomputed or pretrained embeddings. Word2vec, Glove

Word embedding. Example

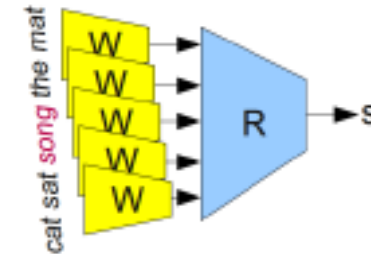
Parametrised function

$$W : \text{words} \rightarrow \mathbb{R}^n$$

Train to predict if a 5 ngram is valid

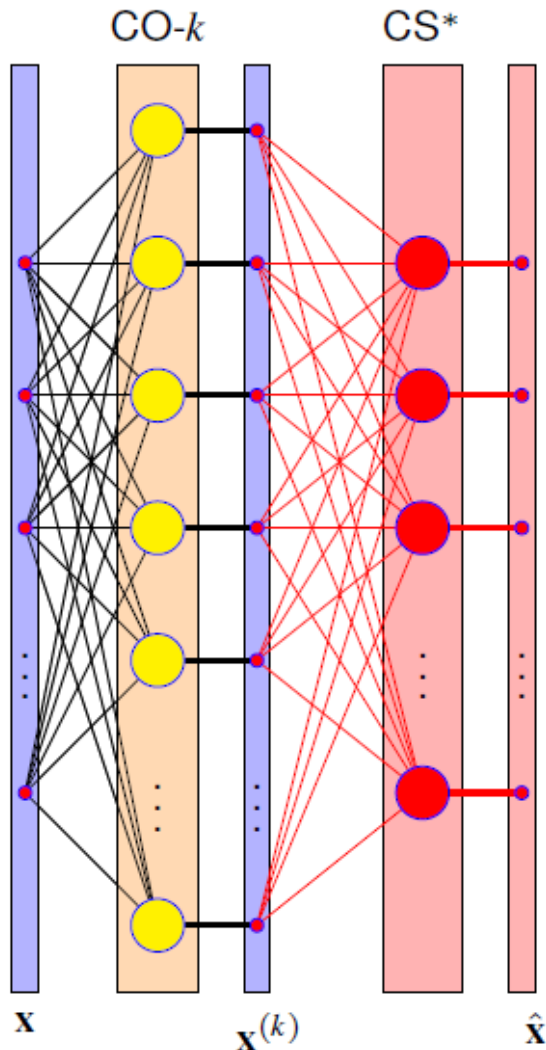
$$R(W(\text{"cat"}), W(\text{"sat"}), W(\text{"on"}), W(\text{"the"}), W(\text{"mat"})) = 1$$

$$R(W(\text{"cat"}), W(\text{"sat"}), W(\text{"song"}), W(\text{"the"}), W(\text{"mat"})) = 0$$



BERT trains by 300 bn tokens to predict the next word

Autoencoder



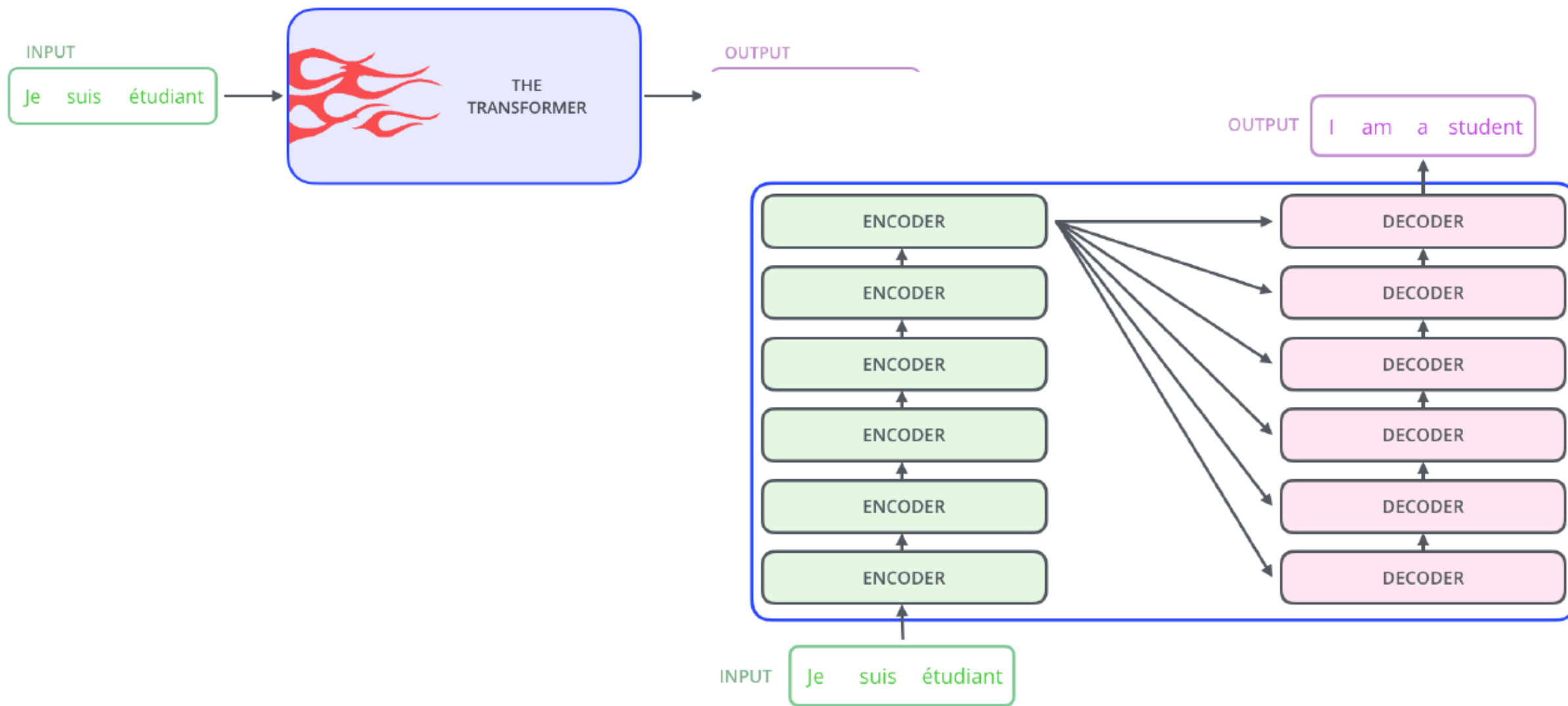
Training in hidden layer to obtain a projection of input with little information loss

- Input $\mathbf{x} = \mathbf{x}^{(k-1)}$
- Desired output $\hat{\mathbf{x}} = \mathbf{x}^{(k-1)}$

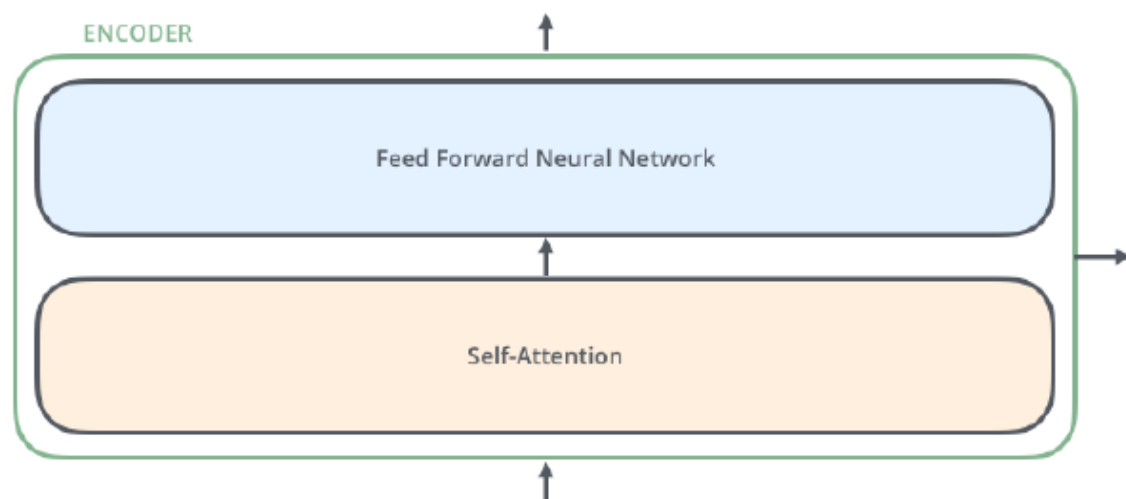
Output layer ‘thrown away’

- Just a mechanism to avoid info loss (projection is reversible)
- Hopefully, we learn relevant features in the process

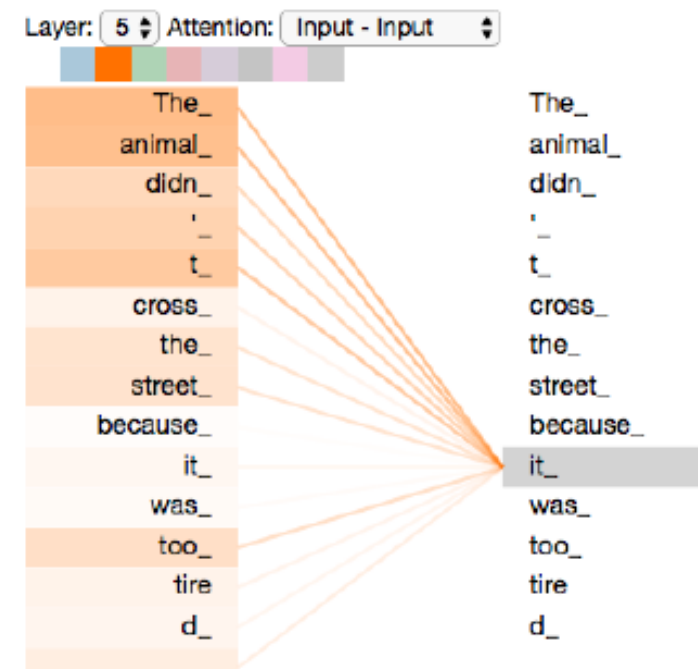
Transformer



Transformer. Encoder



"The animal didn't cross the street because it was too tired"



Transformer. Self-attention

Embedding of i-th token

$$\mathbf{x}_i$$

Query vector

$$\mathbf{q}_i = \mathbf{x}_i' \gamma_q$$

Key vector

$$\mathbf{k}_i = \mathbf{x}_i' \gamma_k$$

Value vector

$$\mathbf{v}_i = \mathbf{x}_i' \gamma_v$$

Output

$$\text{softmax} \left(\frac{\mathbf{q} \mathbf{k}'}{\sqrt{d_k}} \right) \mathbf{v}$$

Input

Thinking

Machines

Embedding

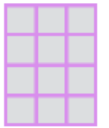
$\mathbf{x}_1$

$\mathbf{x}_2$

Queries

$\mathbf{q}_1$

$\mathbf{q}_2$

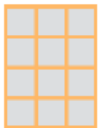


$\mathbf{W}^Q$

Keys

$\mathbf{k}_1$

$\mathbf{k}_2$

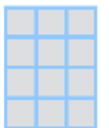


$\mathbf{W}^K$

Values

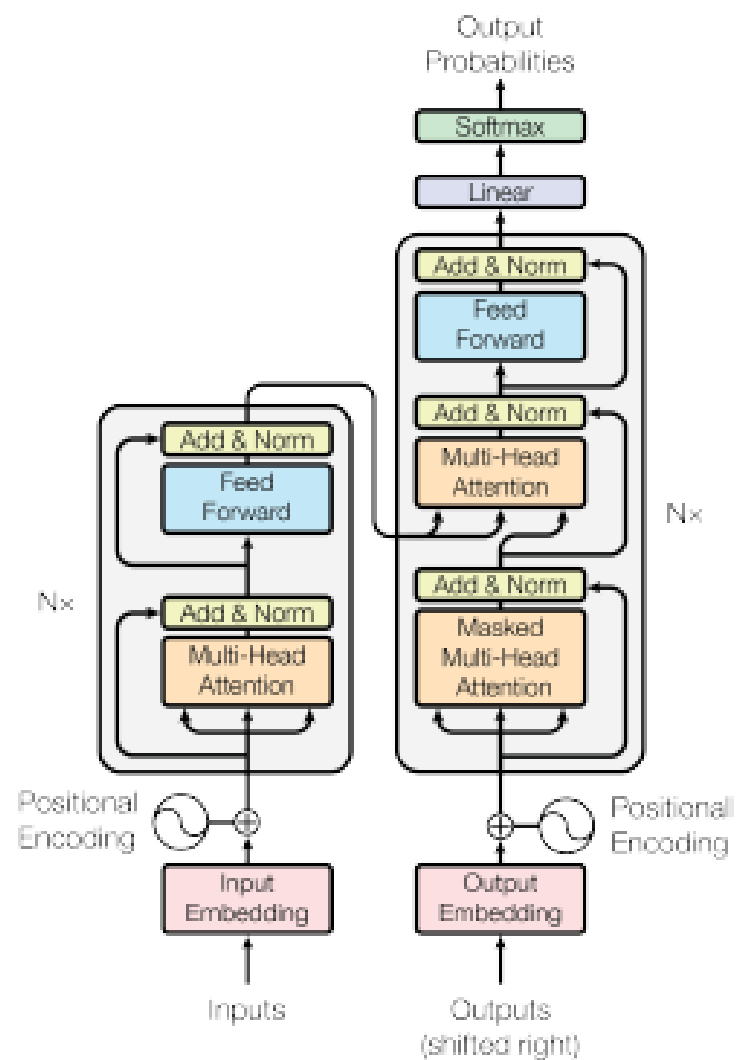
$\mathbf{v}_1$

$\mathbf{v}_2$

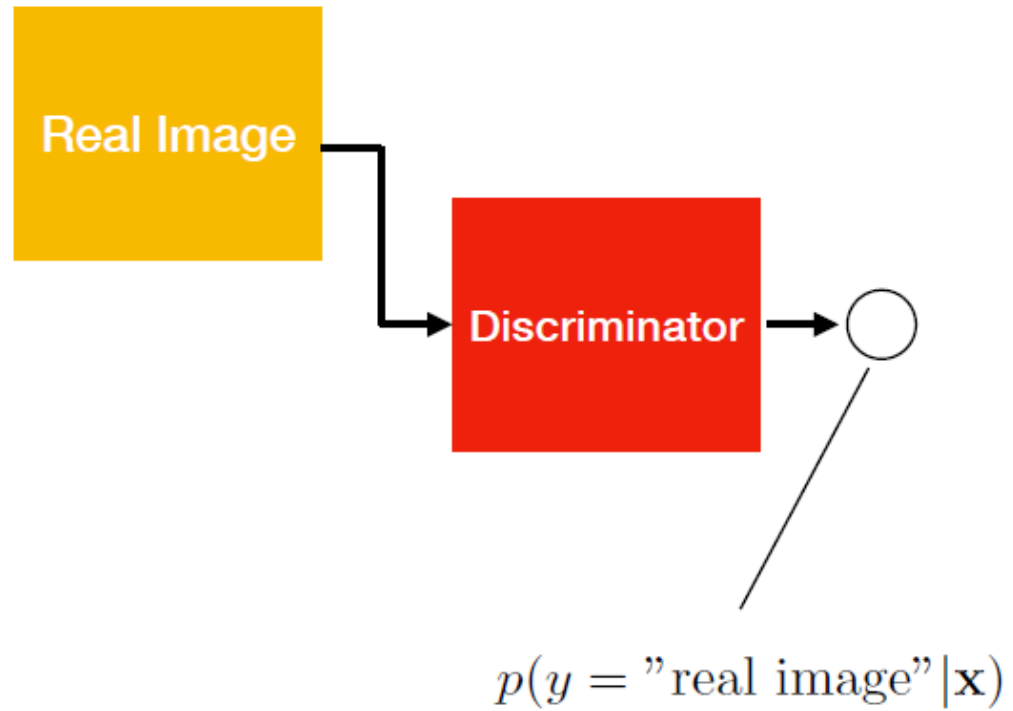


$\mathbf{W}^V$

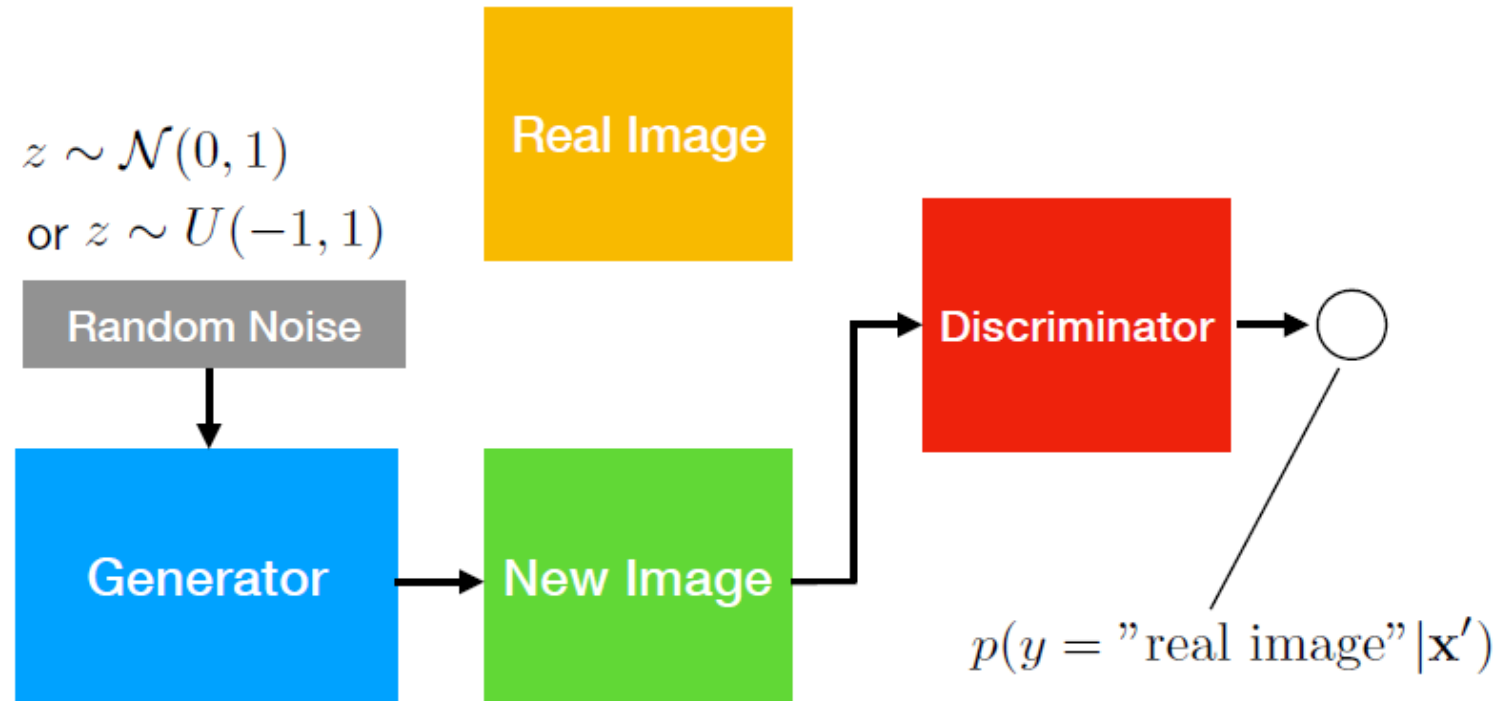
Transformer. Architecture



GANs

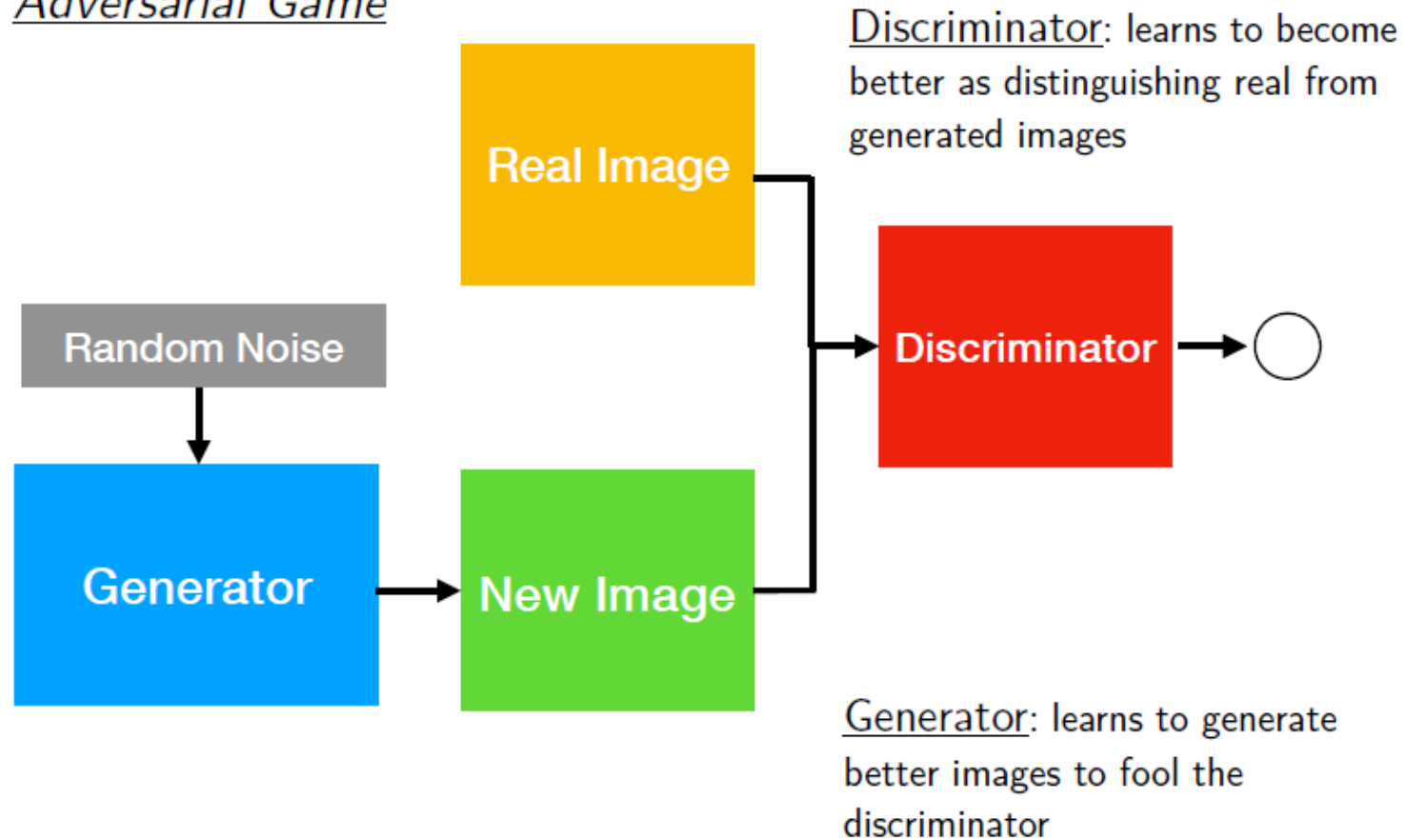


GANs



GANs

Adversarial Game



Towards PML for deep NNs

Towards PML for NNs

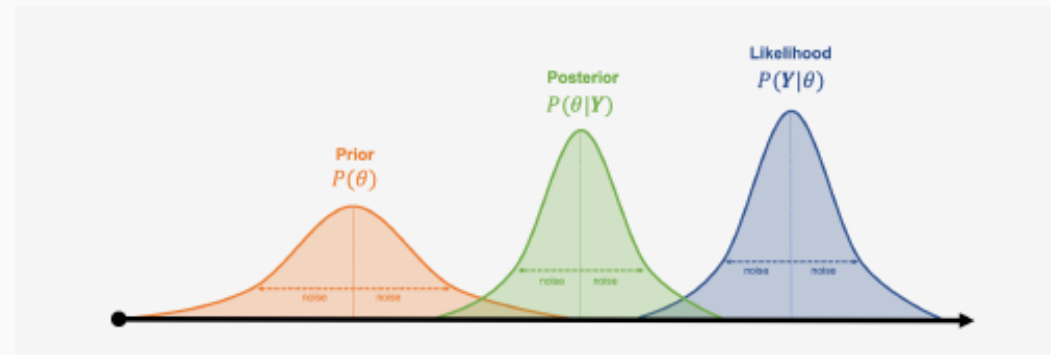
In supervised parametric learning we use a labeled dataset $\mathcal{D} = \{(x_n, y_n) : n = 1 : N\}$ and a probabilistic model $p(y|x, \theta)$.

Deterministic: Prediction is done by calculating $p(y|x, \theta^*)$, where $\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \ell_n(\theta) + \lambda C(\theta)$, where $\ell_n(\theta)$ is a *loss function* which measures goodness of fit, e.g. log-loss $\ell_n(\theta) = -\log p(y|x, \theta)$; $C(\theta)$ is a *regularization factor* and $\lambda \in \mathbb{R}$.

If we use the log-loss, and define $C(\theta) = -\log \pi_0(\theta)$, where $\pi_0(\theta)$ is some prior distribution, and we use $\lambda = 1$, we recover the Maximum a Posteriori (MAP) estimate.

Towards PML for NNs

Bayesian: Instead of predicting using just one value of θ , compute the posterior predictive distribution using Bayesian model averaging $p(\mathbf{y}|\mathbf{x}, D) = \int p(\mathbf{y}|\mathbf{x}, \theta)p(\theta|D)d\theta$. We average over different possible models (possible values of θ), weighted depending on parameter posterior probability $p(\theta|D)$, where $p(\theta|D) \propto p(\theta)p(D|\theta)$, that is, the posterior is proportional to the likelihood times the prior.



Towards PML for NNs

$$p(\mathbf{y}|\mathbf{x}, D) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})p(\boldsymbol{\theta}|D)d\boldsymbol{\theta}.$$
$$p(\boldsymbol{\theta}|D) = \frac{p(\boldsymbol{\theta})p(D|\boldsymbol{\theta})}{\int p(\boldsymbol{\theta})p(D|\boldsymbol{\theta})d\boldsymbol{\theta}}$$

Pros:

- We define a distribution over parameters, which lets us represent uncertainty more accurately.
- Considering different models together can lead to improved accuracy

Cons:

- Integration has no closed form in general
- Problem is usually intractable

Towards PML for NNs

What if we need more complex modelling?

Idea: Leverage deep neural networks (DNNs) for modeling the conditional distribution.

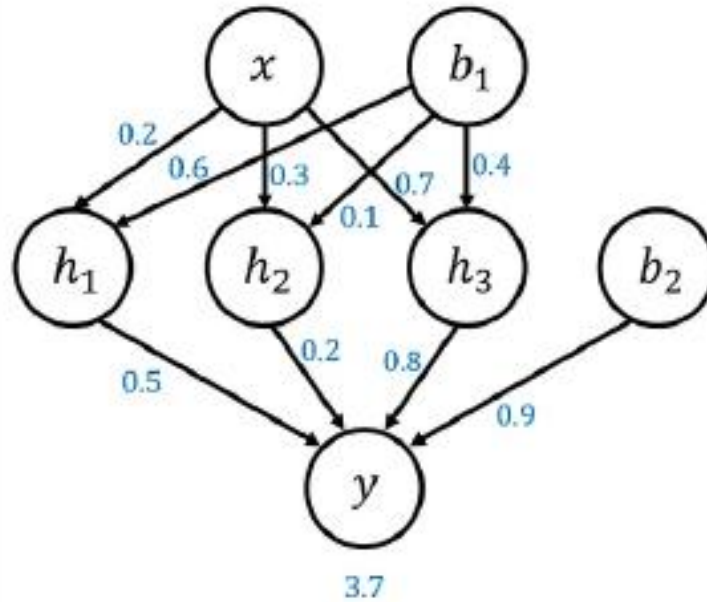
$$p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}) = f_{NN}(\boldsymbol{\theta}; \mathbf{x})$$

But DNNs are large flexible models which can represent many functions, corresponding to different parameter settings (**underspecification**, [D'A+22]), which fit the training data well, yet generalize in different ways. We can benefit from Bayesian model averaging for improved accuracy and uncertainty quantification.

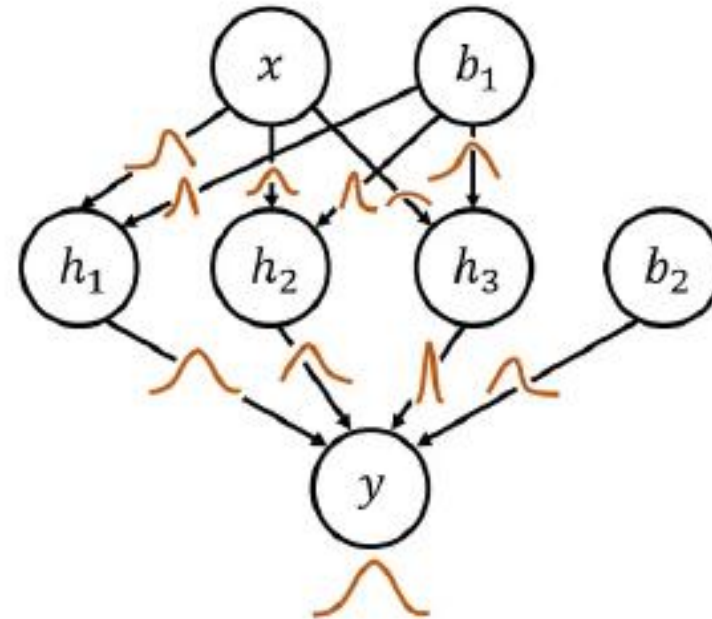
$$p(\mathbf{y}|\mathbf{x}, D) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})p(\boldsymbol{\theta}|D)d\boldsymbol{\theta}$$

Towards PML for NNs

Standard Neural Network



Bayesian Neural Network



Towards PML for NNs

BNNs inherit the ingredients for Bayesian inference:

- Prior parameter distribution $p(\boldsymbol{\theta})$
- Likelihood distribution $p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})$
- Posterior distribution $p(\boldsymbol{\theta}|D)$
- Predictive posterior distribution $p(\mathbf{y}|\mathbf{x}, D)$

Our model becomes more general, but that comes at a cost. We lose convexity, as NNs have nonlinear transformations, and there is a dimensionality explosion, which increases expressivity but compromises inference.

$$p(\mathbf{y}|\mathbf{x}, D) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})p(\boldsymbol{\theta}|D)d\boldsymbol{\theta}.$$

$$p(\boldsymbol{\theta}|D) = \frac{p(\boldsymbol{\theta})p(D|\boldsymbol{\theta})}{\int p(\boldsymbol{\theta})p(D|\boldsymbol{\theta})d\boldsymbol{\theta}}$$

Towards PML for NNs

Nearly all approaches in Bayesian deep learning for estimating the integral

$$p(\mathbf{y}|\mathbf{x}, D) = \int p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta})p(\boldsymbol{\theta}|D)d\boldsymbol{\theta} \quad (1)$$

when it cannot be computed in closed form, involve a *simple Monte Carlo* approximation

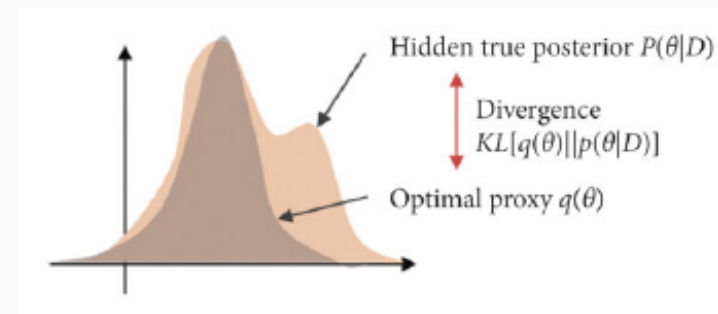
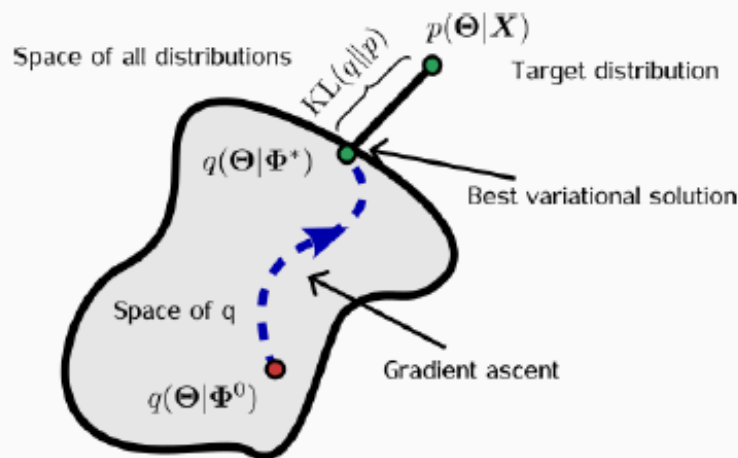
$$p(\mathbf{y}|\mathbf{x}, D) \approx \frac{1}{J} \sum_{j=1}^J p(\mathbf{y}|\mathbf{x}, \boldsymbol{\theta}_j) \quad \boldsymbol{\theta}_j \sim p(\boldsymbol{\theta}|D) \quad (2)$$

This involves samples from the posterior, which in practice will also be approximate as the true posterior will be too complicated to sample from.

Towards PML of NNs

Variational Inference [HVC93; Blu+15]: For posterior approximation of $p(\theta|D)$, choose the distribution $q_\psi(\theta)$ that minimizes $D_{KL}(q_\psi(\theta)||p(\theta|D))$ with respect to ψ .

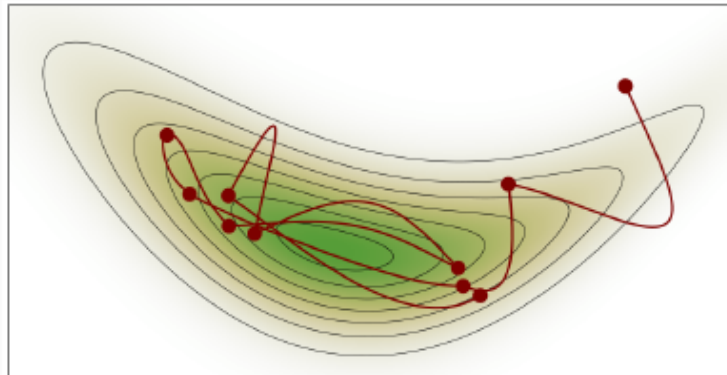
We often choose a Gaussian approximate posterior $q_\psi(\theta) = \mathcal{N}(\theta|\mu, \Sigma)$, which lets us use the reparametrization trick to create a low variance estimator of the gradient of the ELBO.



Towards PML of NNs

MCMC methods [Nea96]: **Hamiltonian Monte Carlo** is considered the gold standard, as it does not make strong assumptions about the form of the posterior.

However, its use is limited because it requires access to the full training set at each step. Stochastic gradient MCMC methods offer a more scalable alternative as they operate using mini-batches of data (e.g. **SGLD**).



Refs and Labs

- Murphy (2023) PMI
- Muller and DRI (2000) Neural Computation
- Gallego and DRI (2022) Annual Review of Stats

- Labs 5. Bayesian neural nets in R